Lab #8\Graphics_in_Seaborn.py

```python
"""
Lab 8 - Visualización: 8 gráficos Matplotlib + 12 gráficos Seaborn
Estructura: Tipo | Propósito | Audiencia
"""
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ============================================================================
# CONFIGURACIÓN Y CARGA DE DATOS (una sola vez)
# ============================================================================
ruta_script = os.path.dirname(os.path.abspath(__file__))
ruta_csv = os.path.join(ruta_script, "ventas_simuladas.csv")
df = pd.read_csv(ruta_csv, encoding="utf-8")

df["Ventas Totales"] = df["Cantidad"] * df["Precio Unitario"]
df["Fecha"] = pd.to_datetime(df["Fecha"], errors="coerce")
df["Mes"] = df["Fecha"].dt.month
df["Año"] = df["Fecha"].dt.year
df["Dia Semana"] = df["Fecha"].dt.day_name()

map_categoria = {
    "Leche": "Lácteos", "Pan": "Panadería", "Café": "Bebidas calientes",
    "Té": "Bebidas calientes", "Chocolate": "Bebidas calientes",
    "Cereal": "Cereales", "Galletas": "Galletas y snacks", "Azúcar": "Endulzantes"
}
df["Categoria"] = df["Producto"].map(map_categoria).fillna("Otros")

NOMBRES_MES = {1: "Ene", 2: "Feb", 3: "Mar", 4: "Abr", 5: "May", 6: "Jun",
               7: "Jul", 8: "Ago", 9: "Sep", 10: "Oct", 11: "Nov", 12: "Dic"}
fig_std, color_bar = (8, 6), "#48C9B0"


def _etiquetas_barras_h(ax, fmt=",.0f"):
    for p in ax.patches:
        w = p.get_width()
        ax.text(w + 5, p.get_y() + p.get_height() / 2, f"{w:{fmt}}", va="center", fontsize=10, fontweight="bold")


def _etiquetas_barras_v(ax, fmt=",.0f"):
    for p in ax.patches:
        h = p.get_height()
        ax.text(p.get_x() + p.get_width() / 2, h + 5, f"{h:{fmt}}", ha="center", va="bottom", fontsize=10, fontweight="bold")


# ============================================================================
# 8 GRÁFICOS CON MATPLOTLIB
# ============================================================================

```

```python
# --- MAT 1: Step Chart (Gráfico de gradas) ---
# Tipo: Líneas escalonadas
# Propósito: Mostrar evolución de ventas acumuladas en el tiempo (crecimiento acumulativo).
# Audiencia: Gerencia, equipo financiero, inversionistas.
ventas_fecha = df.groupby("Fecha")["Ventas Totales"].sum().reset_index().sort_values("Fecha")
ventas_fecha["Acumulado"] = ventas_fecha["Ventas Totales"].cumsum()
fig, ax = plt.subplots(figsize=(10, 5))
ax.step(ventas_fecha["Fecha"], ventas_fecha["Acumulado"], where="post", color="steelblue", linewidth=2)
ax.set_xlabel("Fecha")
ax.set_ylabel("Ventas acumuladas")
ax.set_title("Gráfico de gradas: Ventas acumuladas en el tiempo")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "MAT_01_step.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- MAT 2: Funnel Chart (Gráfico de embudo) ---
# Tipo: Barras horizontales proporcionales (embudo)
# Propósito: Comparar ventas entre vendedores; identificar quién aporta más.
# Audiencia: Supervisores de ventas, gerencia comercial, RR.HH.
dat = df.groupby("Vendedor")["Ventas Totales"].sum().sort_values(ascending=False).reset_index()
fig, ax = plt.subplots(figsize=fig_std)
y_pos = range(len(dat))
anchos = dat["Ventas Totales"] / dat["Ventas Totales"].max()
ax.barh(y_pos, anchos, left=(1 - anchos) / 2, height=0.7, color="steelblue", edgecolor="white",
linewidth=1)
ax.set_yticks(y_pos)
ax.set_yticklabels(dat["Vendedor"])
ax.set_xlim(0, 1)
ax.set_xticks([])
ax.set_title("Funnel chart: Ventas por vendedor")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "MAT_02_funnel_vendedor.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- MAT 3: Histograma ---
# Tipo: Histograma
# Propósito: El histograma revela si el negocio depende de muchas transacciones pequeñas
# o de unas pocas ventas grandes.
# Audiencia: Gerencia de Operaciones. Les sirve para planificar el flujo de caja y la
# cantidad de pedidos que deben procesar.
fig, ax = plt.subplots(figsize=(10, 6))
ax.hist(df["Ventas Totales"], bins=15, color=color_bar, edgecolor="black", alpha=0.7)
ax.set_title("Distribución de los Montos de Ventas Totales", fontweight="bold")
ax.set_xlabel("Monto de la Venta ($)")
ax.set_ylabel("Frecuencia")
ax.grid(axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "MAT_03_histograma.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- MAT 4: Boxplot ---
# Tipo: Boxplot
# Propósito: Muestra el control de estabilidad de precios en los que se mueve cada producto.
```

```python
# Audiencia: Auditores o Analistas Financieros. Ayuda a detectar anomalías en precios o
# asegurar que los márgenes de ganancia se mantengan.
productos = df["Producto"].unique()
datos_bp = [df[df["Producto"] == p]["Precio Unitario"].values for p in productos]
fig, ax = plt.subplots(figsize=(10, 6))
ax.boxplot(datos_bp, tick_labels=productos, patch_artist=True,
           boxprops=dict(facecolor="#a2d2ff", color="blue"), medianprops=dict(color="red"))
ax.set_title("Análisis de Precios Unitarios por Categoría de Producto", fontweight="bold")
ax.set_xlabel("Producto")
ax.set_ylabel("Precio Unitario ($)")
ax.grid(axis="y", linestyle=":", alpha=0.6)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "MAT_04_boxplot.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- MAT 5: Barras horizontales (ventas por producto) ---
# Tipo: Barras horizontales
# Propósito: Esta gráfica muestra barras horizontales que permiten determinar los productos
# que más se han vendido según la venta total.
# Audiencia: Equipo de ventas para evaluar resultados y tomar decisiones con relación a
# qué producto necesita incrementar su venta y cuál es el "producto estrella". También
# equipo de abastecimiento estratégico para determinar la importancia en la cadena de suministros.
# Agrupar por producto y sumar ventas
dat = df.groupby("Producto", as_index=False)["Ventas Totales"].sum().sort_values("Ventas Totales",
ascending=True)
fig, ax = plt.subplots(figsize=(10, 6))
bars = ax.barh(dat["Producto"], dat["Ventas Totales"], color=color_bar)
ax.set_title("Ventas Totales por Producto")
ax.set_xlabel("Ventas Totales")
ax.set_ylabel("Producto")
_etiquetas_barras_h(ax)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "MAT_05_barras_producto.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- MAT 6: Barras verticales (ventas por vendedor) ---
# Tipo: Barras verticales (columnas)
# Propósito: Esta gráfica permite ver las ventas totales por cada vendedor con el objetivo
# de determinar oportunidades de mejora con los vendedores más bajos y evidenciar el buen
# desempeño de los vendedores más altos.
# Audiencia: Equipo de ventas para evaluar el desempeño del equipo.
# Agrupar por vendedor y sumar ventas
dat = df.groupby("Vendedor", as_index=False)["Ventas Totales"].sum().sort_values("Ventas Totales",
ascending=False)
fig, ax = plt.subplots(figsize=(12, 6))
bars = ax.bar(dat["Vendedor"], dat["Ventas Totales"], color=color_bar)
ax.set_title("Ventas Totales por Vendedor")
ax.set_xlabel("Vendedor")
ax.set_ylabel("Ventas Totales")
plt.xticks(rotation=45, ha="right")
_etiquetas_barras_v(ax)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "MAT_06_barras_vendedor.png"), dpi=150, bbox_inches="tight")
plt.show()
```

```python

# --- MAT 7: Stackplot (barras apiladas) ---
# Tipo: Área apilada
# Propósito: Este gráfico permite ver qué categoría tiene mayor aporte del total de las
# ventas en un año.
# Audiencia: Equipo comercial estratégico que pueda utilizar la información para diseñar
# nuevas propuestas de ventas para el próximo periodo.
# Filtrar año de referencia y agrupar por mes y categoría
año_ref = df["Año"].max()
df_año = df[df["Año"] == año_ref].copy()
df_año["Mes"] = df_año["Fecha"].dt.month
dat = df_año.groupby(["Mes", "Categoria"])["Ventas Totales"].sum().reset_index()
# Convertir a formato para stackplot (pivot)
pivot = dat.pivot(index="Mes", columns="Categoria", values="Ventas Totales").fillna(0).sort_index()
fig, ax = plt.subplots(figsize=(12, 6))
ax.stackplot(pivot.index, pivot.T.values, labels=pivot.columns)
ax.set_title(f"Ventas Totales por Mes y Categoría - {año_ref}")
ax.set_xlabel("Mes")
ax.set_ylabel("Ventas Totales")
ax.set_xticks(range(1, 13), [NOMBRES_MES[i] for i in range(1, 13)])
ax.legend(title="Categoría", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "MAT_07_stackplot.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- MAT 8: Gráfico de líneas (tendencia mensual) ---
# Tipo: Gráfico de líneas
# Propósito: Muestra cómo evolucionaron las ventas totales mes a mes. Permite detectar
# picos, caídas y patrones estacionales que se repiten entre años.
# Audiencia: Gerentes de ventas, directores comerciales y analistas de negocio que
# necesiten planificar estrategias, promociones o niveles de inventario según el historial.
# Se agrupa por año y mes, se suman todas las ventas totales de cada mes
ventas_mes = df.groupby(["Año", "Mes"])["Ventas Totales"].sum().reset_index()
fig, ax = plt.subplots(figsize=(10, 6))
# Con unique obtenemos los años diferentes que existen
for año in ventas_mes["Año"].unique():
    d = ventas_mes[ventas_mes["Año"] == año]  # Filtra los datos solo para ese año
    ax.plot(d["Mes"], d["Ventas Totales"], marker="o", label=str(int(año)))  # marker: punto en cada mes
ax.set_title("Tendencia mensual de ventas")
ax.set_xlabel("Mes")
ax.set_ylabel("Ventas Totales ($)")
ax.legend()
ax.grid(True)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "MAT_08_tendencia_mensual.png"), dpi=150, bbox_inches="tight")
plt.show()

# ==============================================================================
# 12 GRÁFICOS CON SEABORN
# ==============================================================================
sns.set_theme(style="whitegrid", palette="muted")

# --- SNS 1: Scatter (dispersión) ---
```

```python
# Tipo: Gráfico de dispersión
# Propósito: Explorar relación entre precio y cantidad; ver si hay patrones según promoción.
# Audiencia: Equipo de marketing, analistas de precios, estrategia comercial.
fig, ax = plt.subplots(figsize=fig_std)
sns.scatterplot(data=df, x="Precio Unitario", y="Cantidad", hue="Promoción", ax=ax)
ax.set_title("Cantidad vs Precio por Promoción")
ax.legend(title="Promoción")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_01_scatter.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 2: Relplot ---
# Tipo: Gráfico de líneas por facetas
# Propósito: Comparar evolución de ventas por mes entre diferentes años.
# Audiencia: Planificación, gerencia, análisis de tendencias estacionales.
ventas_ma = df.groupby(["Mes", "Año"])["Ventas Totales"].sum().reset_index()
g = sns.relplot(data=ventas_ma, x="Mes", y="Ventas Totales", col="Año", hue="Año",
                kind="line", palette="crest", linewidth=2, col_wrap=2, height=2.5, aspect=1.2, legend=False)
g.set_titles("Año {col_name}")
g.set_axis_labels("Mes", "Ventas")
plt.tight_layout()
g.savefig(os.path.join(ruta_script, "SNS_02_relplot.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 3: Funnel categorías ---
# Tipo: Gráfico de embudo (barras proporcionales)
# Propósito: Comparar ventas por categoría; la amplitud muestra el peso de cada una.
# Audiencia: Compras, gerencia de producto, estrategia comercial.
dat = df.groupby("Categoria")["Ventas Totales"].sum().sort_values(ascending=False).reset_index()
fig, ax = plt.subplots(figsize=fig_std)
y_pos = range(len(dat))
anchos = dat["Ventas Totales"] / dat["Ventas Totales"].max()
ax.barh(y_pos, anchos, left=(1 - anchos) / 2, height=0.7,
        color=sns.color_palette("Blues_r", len(dat)), edgecolor="white", linewidth=1)
ax.set_yticks(y_pos)
ax.set_yticklabels(dat["Categoria"])
ax.set_xlim(0, 1)
ax.set_xticks([])
ax.set_title("Funnel chart: Ventas por categoría")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_03_funnel_categorias.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 4: Residplot ---
# Tipo: Gráfico de residuos de regresión lineal
# Propósito: Verificar ajuste del modelo lineal entre precio y ventas; detectar patrones
# en los errores.
# Audiencia: Analistas de datos, equipo de predicción, estadística.
fig, ax = plt.subplots(figsize=fig_std)
sns.residplot(data=df, x="Precio Unitario", y="Ventas Totales", lowess=True, color="g", ax=ax)
ax.set_title("Residuos: Ventas vs Precio Unitario")
ax.set_ylabel("Residuos")
plt.tight_layout()
```

```python
fig.savefig(os.path.join(ruta_script, "SNS_04_residplot.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 5: Histograma ---
# Tipo: Histograma + KDE
# Propósito: El histograma revela si el negocio depende de muchas transacciones pequeñas
# o de unas pocas ventas grandes.
# Audiencia: Gerencia de Operaciones. Les sirve para planificar el flujo de caja y la
# cantidad de pedidos que deben procesar.
fig, ax = plt.subplots(figsize=(10, 6))
sns.histplot(data=df, x="Ventas Totales", kde=True, color="teal")
ax.set_title("Distribución de Frecuencia de Ventas Totales")
ax.set_xlabel("Monto de Venta ($)")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_05_histograma.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 6: Heatmap (mapa de calor) ---
# Tipo: Mapa de calor
# Propósito: Identifica quién es el líder en cada categoría. Facilita ver si hay productos
# que nadie está vendiendo.
# Audiencia: Dirección Comercial. Útil para juntas de resultados mensuales donde se busca
# evidenciar el rendimiento.
# Preparamos los datos en forma de tabla (pivot)
pivot = df.pivot_table(index="Vendedor", columns="Producto", values="Ventas Totales", aggfunc="sum")
fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(pivot, annot=True, fmt=".1f", cmap="YlGnBu", linewidths=0.5)
ax.set_title("Desempeño por Vendedor y Producto")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_06_heatmap.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 7: Boxplot ---
# Tipo: Boxplot
# Propósito: Muestra el control de estabilidad de precios en los que se mueve cada producto.
# Audiencia: Auditores o Analistas Financieros. Ayuda a detectar anomalías en precios o
# asegurar que los márgenes de ganancia se mantengan.
fig, ax = plt.subplots(figsize=(10, 6))
sns.boxplot(data=df, x="Producto", y="Precio Unitario", hue="Producto", palette="Set2", legend=False)
ax.set_title("Análisis de Dispersión de Precios Unitarios")
ax.set_xlabel("Categoría de Producto")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_07_boxplot.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 8: Barras ventas por promoción ---
# Tipo: Barras horizontales
# Propósito: Este gráfico de barras permite revisar las ventas totales de productos que
# tuvieron o no promoción. El objetivo es determinar si las promociones están teniendo un
# impacto significativo en las ventas totales.
# Audiencia: Equipo de marketing o investigación de mercado que determina las promociones.
# Objetivo: determinar si están aplicando promociones de alto impacto o no.
# Agrupar por promoción y sumar ventas
```

```python
dat = df.groupby("Promoción", as_index=False)["Ventas Totales"].sum().sort_values("Ventas Totales", ascending=False)
fig, ax = plt.subplots(figsize=(12, 6))
sns.barplot(data=dat, x="Ventas Totales", y="Promoción", color=color_bar)
ax.set_title("Ventas Totales por Promoción")
_etiquetas_barras_h(ax)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_08_barras_promocion.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 9: Barras cantidad por mes (categoría) ---
# Tipo: Barras (columnas)
# Propósito: Este gráfico permite evaluar la cantidad vendida de una categoría en específico
# para determinar en qué meses del año se venden más los productos de esta categoría.
# Audiencia: Equipo ejecutivo estratégico que tome decisiones sobre el aumento en la
# producción de ciertos productos.
# Filtrar año y categoría específica, agrupar por mes
cat_ref = "Galletas y snacks"
df_f = df[(df["Año"] == año_ref) & (df["Categoria"] == cat_ref)].copy()
df_f["Mes"] = df_f["Fecha"].dt.month
dat = df_f.groupby("Mes", as_index=False)["Cantidad"].sum().sort_values("Mes")
dat["Mes_nombre"] = dat["Mes"].map(NOMBRES_MES)
fig, ax = plt.subplots(figsize=(12, 6))
sns.barplot(data=dat, x="Mes_nombre", y="Cantidad", color=color_bar)
ax.set_title(f"Cantidad Vendida de {cat_ref} por Mes - {año_ref}")
_etiquetas_barras_v(ax, ".0f")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_09_barras_galletas_mes.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 10: Lineplot día semana ---
# Tipo: Gráfico de líneas
# Propósito: Revela qué días concentran más actividad comercial durante la semana, haciendo
# visibles los picos y valles de demanda de forma clara.
# Audiencia: Gerentes de tienda, equipos de operaciones y marketing para ajustar turnos
# de personal, lanzar promociones en los días clave o reforzar el stock antes de los
# momentos de mayor demanda.
# Ordenar días correctamente (lunes a domingo)
orden_dias = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
ventas_dia = df.groupby("Dia Semana")["Ventas Totales"].sum().reset_index()
ventas_dia["Dia Semana"] = pd.Categorical(ventas_dia["Dia Semana"], categories=orden_dias, ordered=True)
ventas_dia = ventas_dia.sort_values("Dia Semana")
fig, ax = plt.subplots(figsize=fig_std)
sns.lineplot(data=ventas_dia, x="Dia Semana", y="Ventas Totales", marker="o")
ax.set_title("Tendencia de ventas por día de la semana")
ax.set_xlabel("Día de la semana")
plt.xticks(rotation=45)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_10_lineplot_dia_semana.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 11: Violin vendedor ---
# Tipo: Gráfico de violín
```

```python
# Propósito: Compara el rendimiento y la variabilidad de ventas de cada integrante del
# equipo. Muestra si el desempeño es parejo o si hay vendedores con resultados más inconsistentes.
# Audiencia: Gerentes y supervisores de ventas para tener una visión comparativa del equipo
# e identificar quién tiene mayor consistencia y quién presenta más altibajos.
fig, ax = plt.subplots(figsize=fig_std)
sns.violinplot(data=df, x="Vendedor", y="Ventas Totales")
ax.set_title("Distribución de ventas por vendedor")
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_11_violin_vendedor.png"), dpi=150, bbox_inches="tight")
plt.show()

# --- SNS 12: Countplot productos ---
# Tipo: Barras de conteo
# Propósito: Muestra la frecuencia de transacciones por producto y según promoción.
# Audiencia: Operaciones, inventario, equipo de ventas.
fig, ax = plt.subplots(figsize=(10, 6))
sns.countplot(data=df, x="Producto", hue="Promoción", palette="Set2")
ax.set_title("Cantidad de transacciones por Producto y Promoción")
ax.set_xlabel("Producto")
plt.xticks(rotation=45)
plt.tight_layout()
fig.savefig(os.path.join(ruta_script, "SNS_12_countplot.png"), dpi=150, bbox_inches="tight")
plt.show()
```

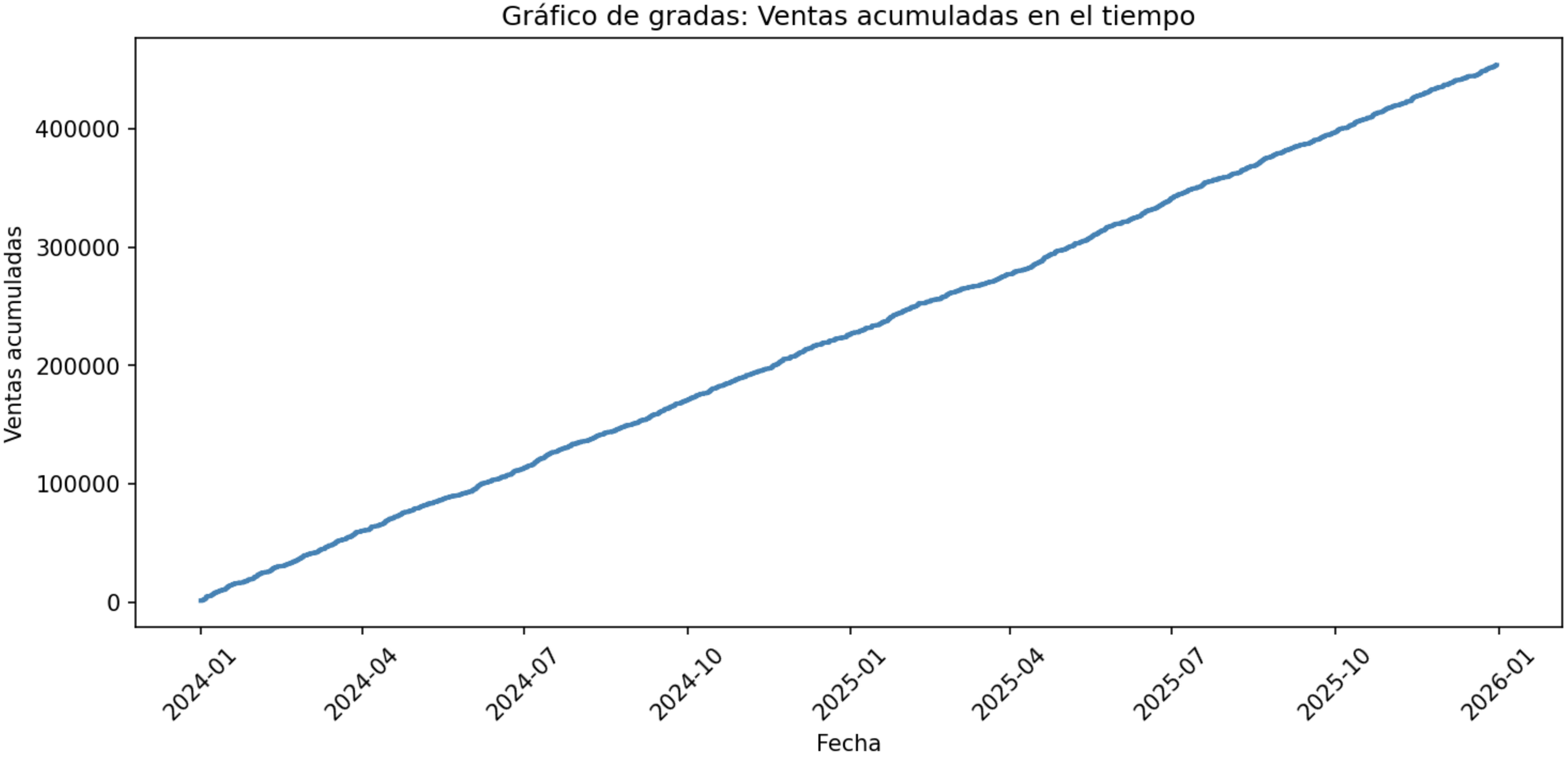
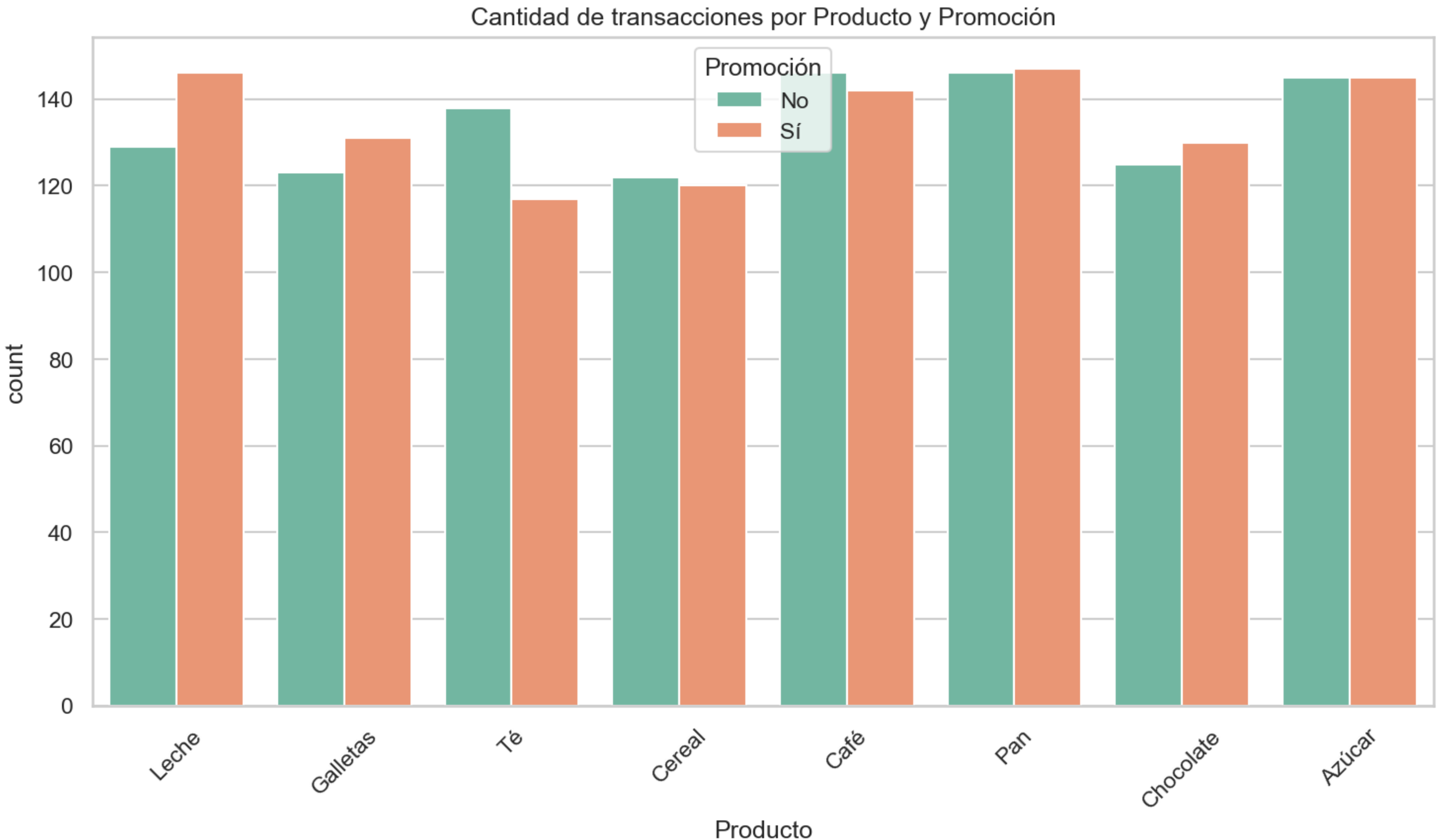
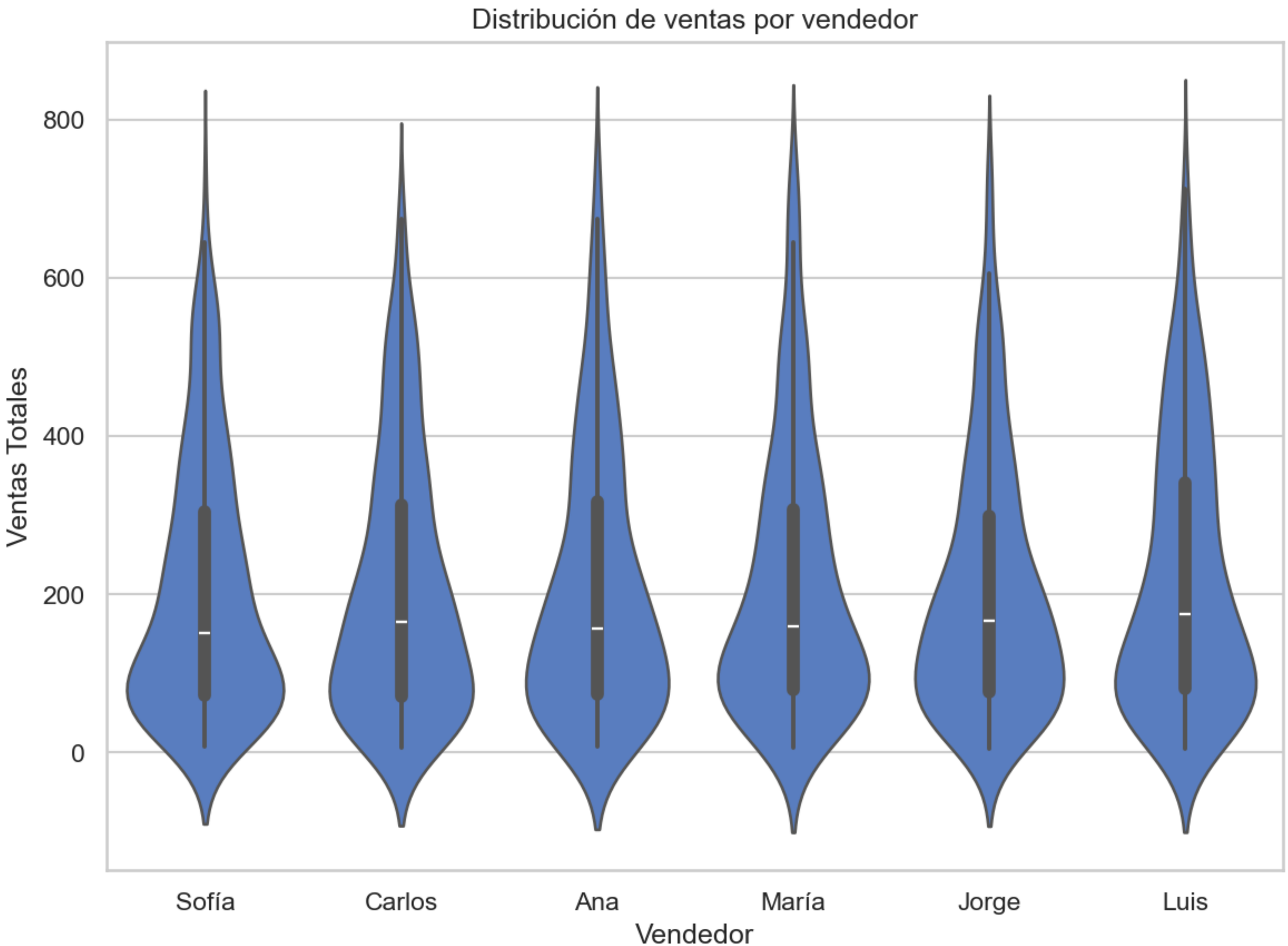
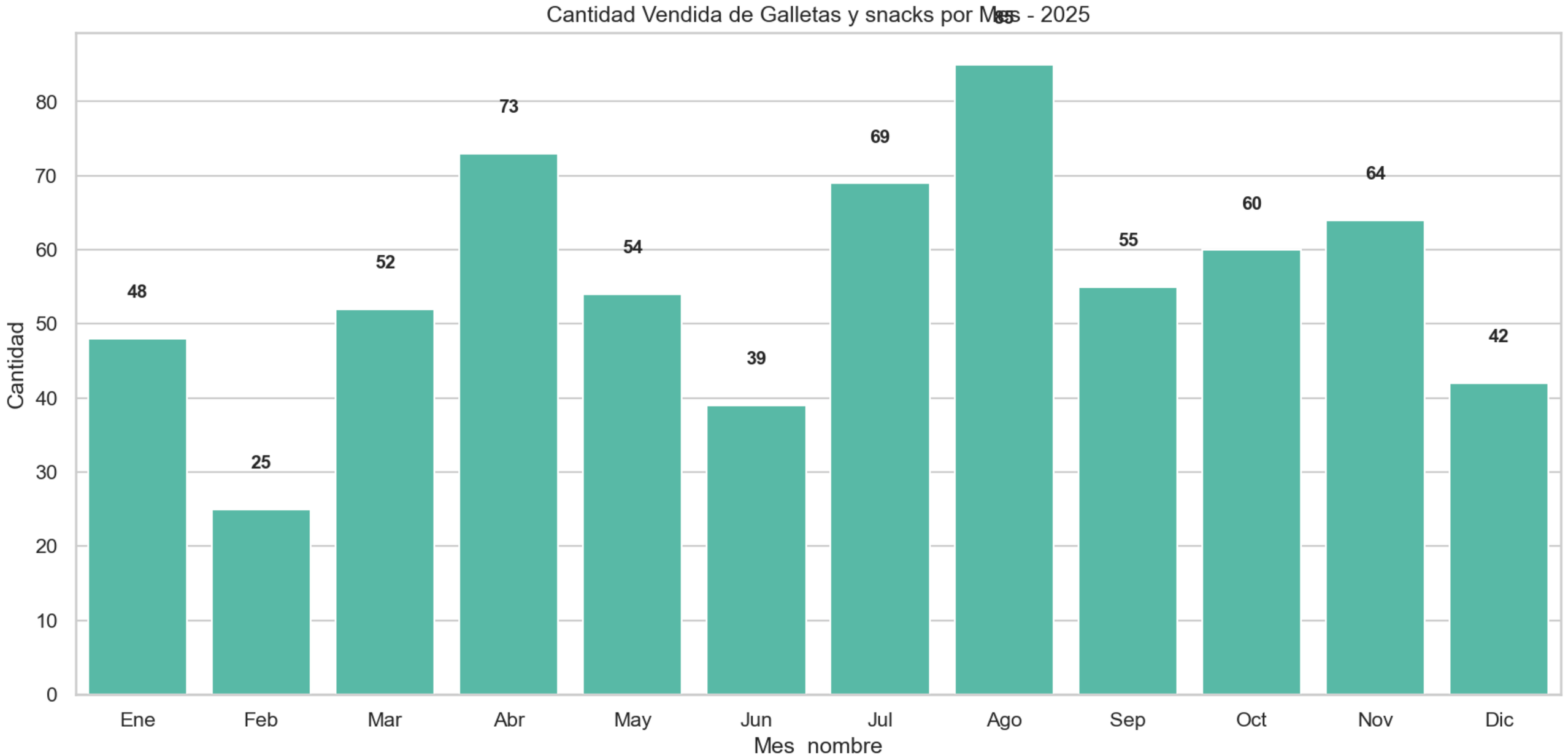


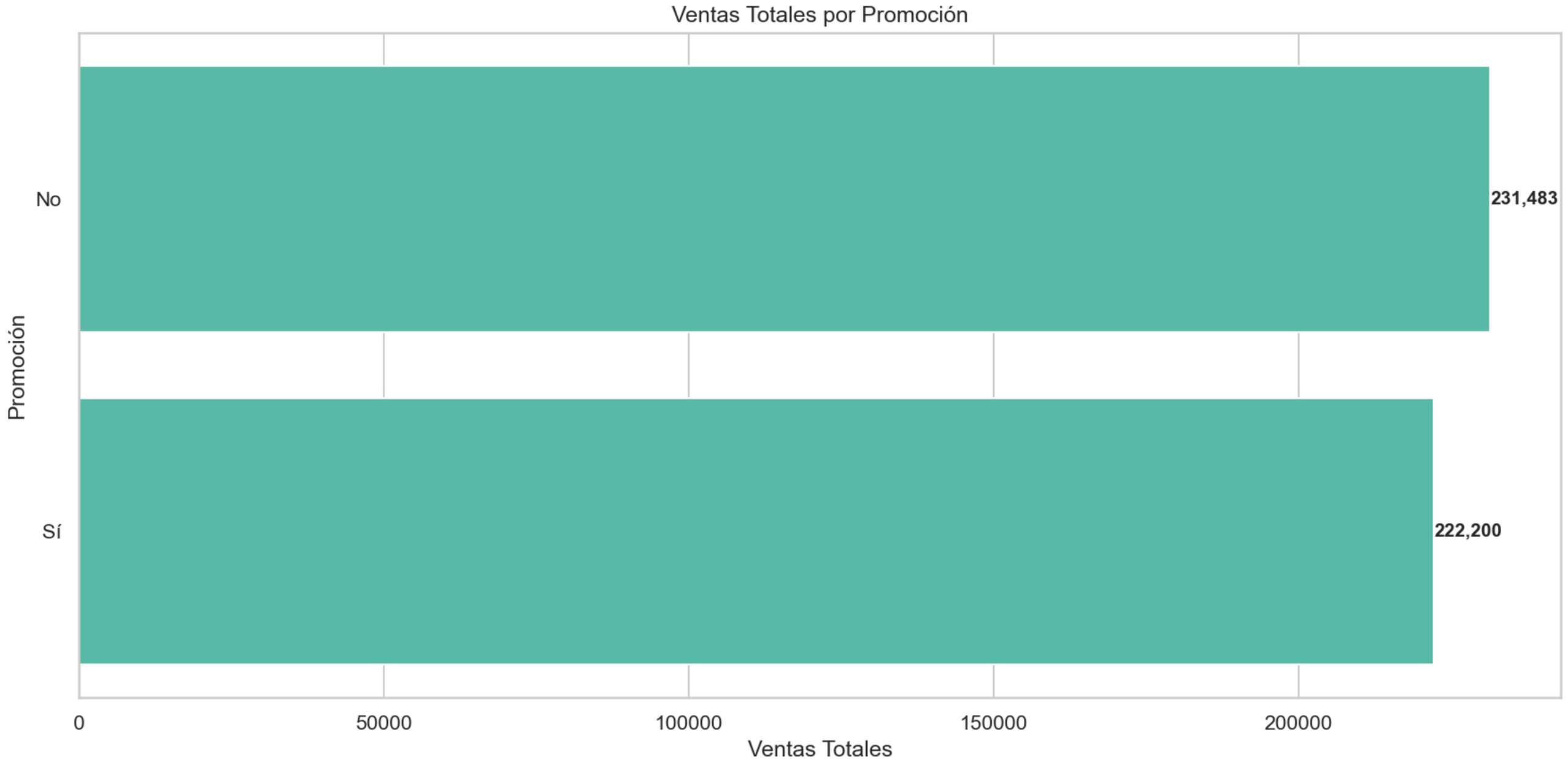
Gráfico de gradas: Ventas acumuladas en el tiempo
Ventas acumuladas
400000
300000
200000
100000
0
2024-01
2024-04
2024-07
2024-10
2025-01
2025-04
2025-07
2025-10
2026-01
Fecha

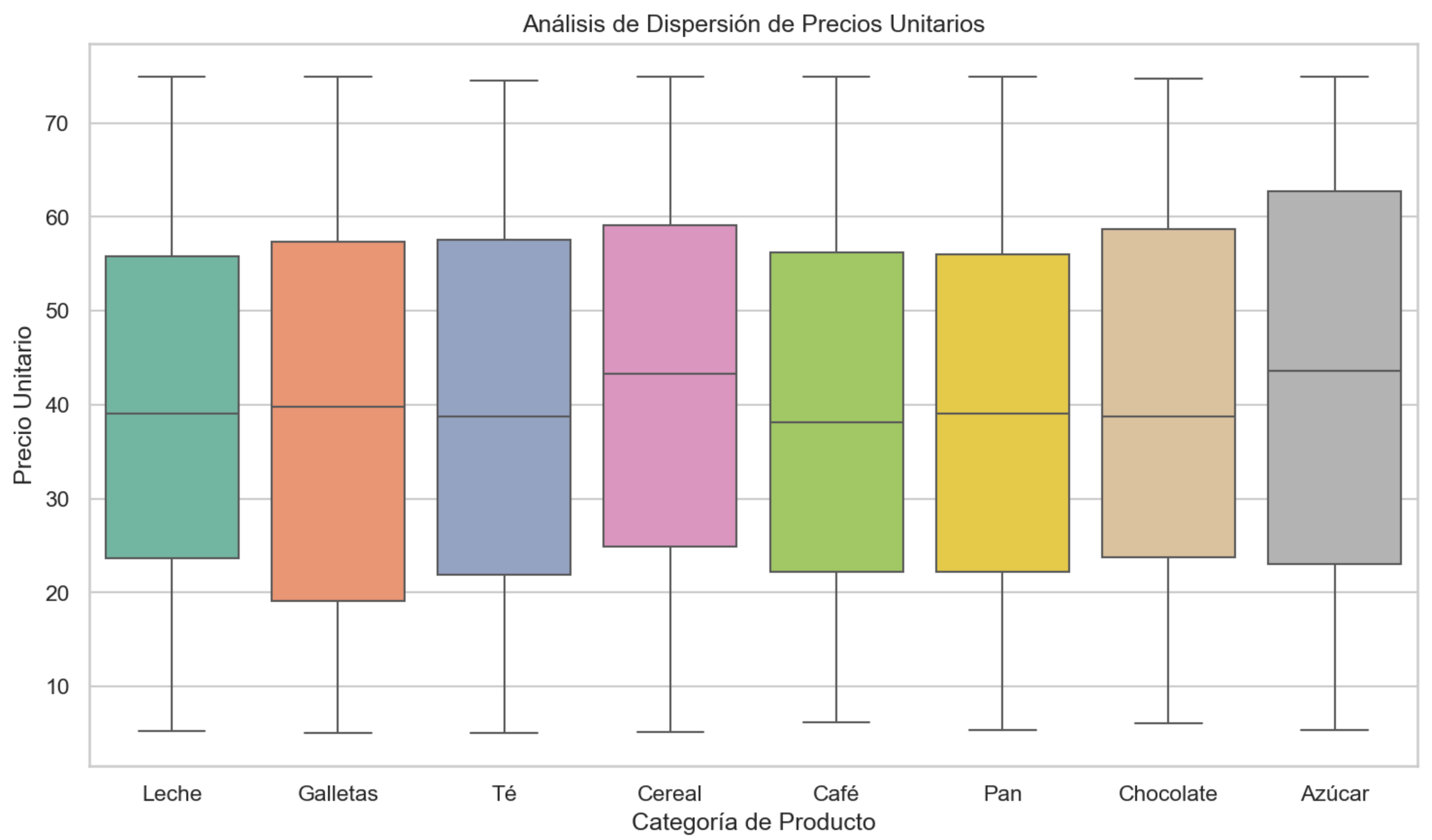

Cantidad de transacciones por Producto y Promoción
Promoción
No
Sí
count
0
20
40
60
80
100
120
140
Leche
Galletas
Té
Cereal
Café
Pan
Chocolate
Azúcar
Producto

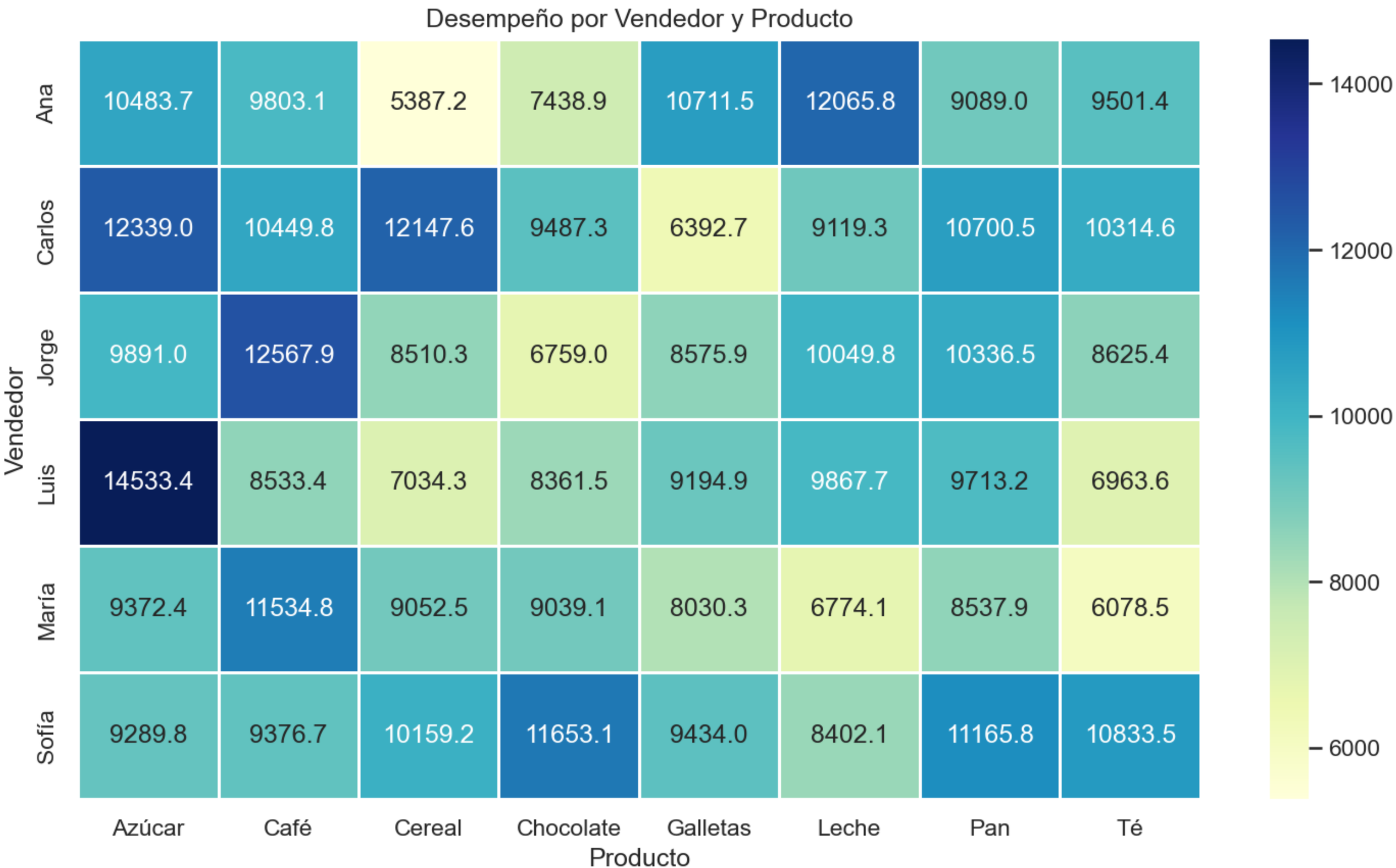

Distribución de ventas por vendedor
Ventas Totales
800
600
400
200
0
Sofía
Carlos
Ana
María
Jorge
Luis
Vendedor

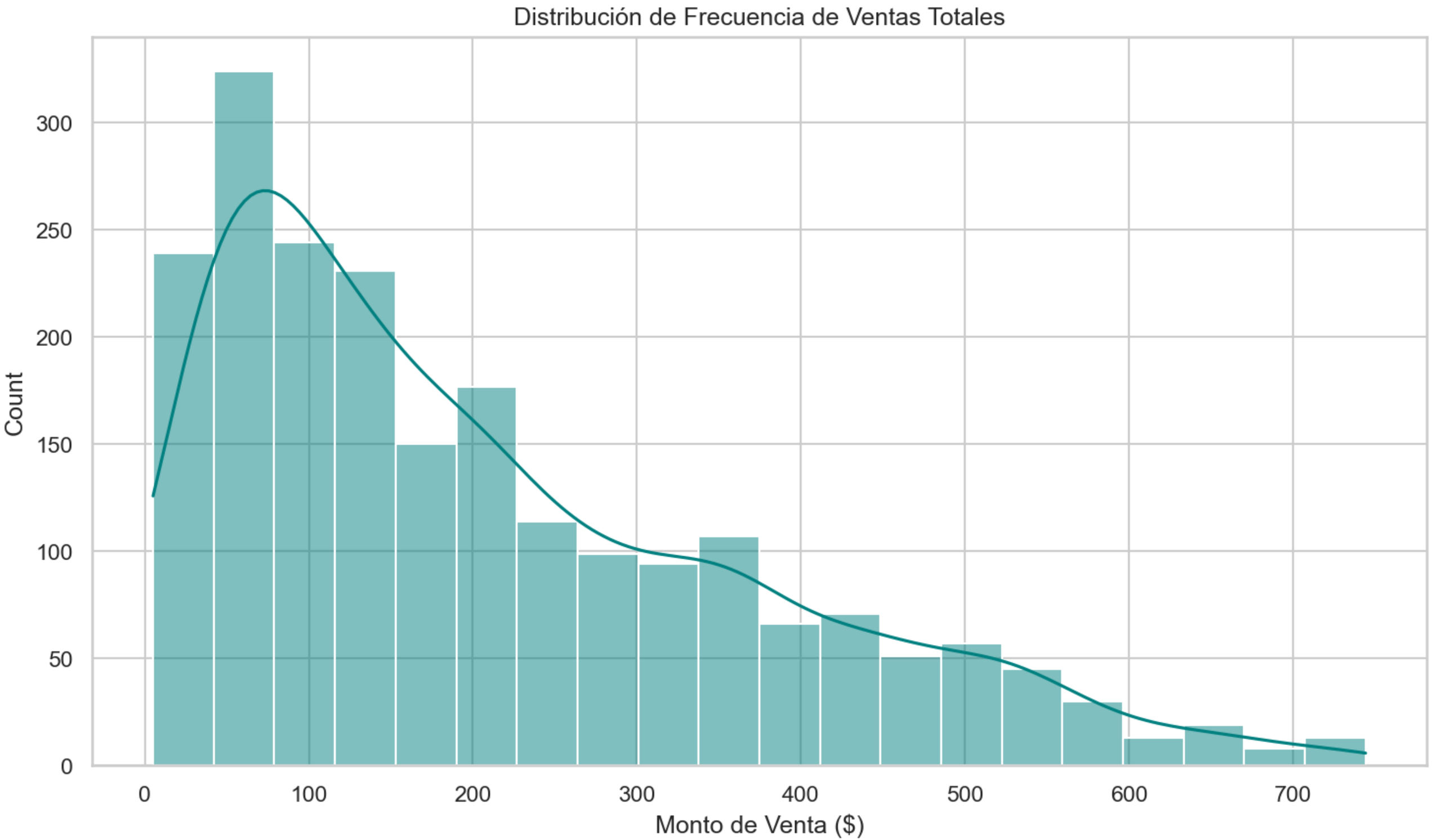
Tendencia de ventas por día de la semana
70000
68000
66000
64000
62000
60000
Ventas Totales
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
Día de la semana

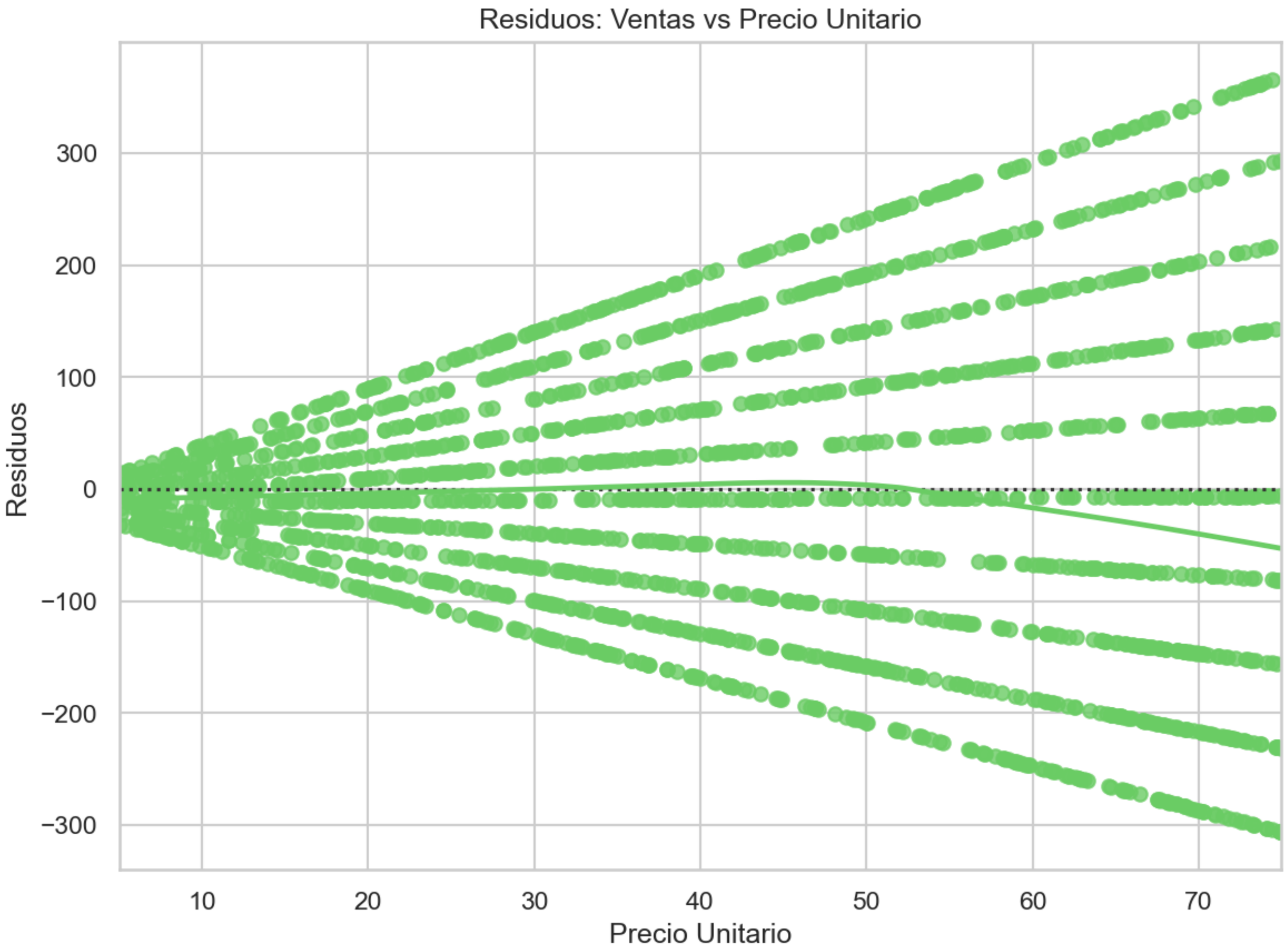

Cantidad Vendida de Galletas y snacks por Mes - 2025
Cantidad
Mes_nombre
0
10
20
30
40
50
60
70
80
48
25
52
73
54
39
69
85
55
60
64
42
Ene
Feb
Mar
Abr
May
Jun
Jul
Ago
Sep
Oct
Nov
Dic

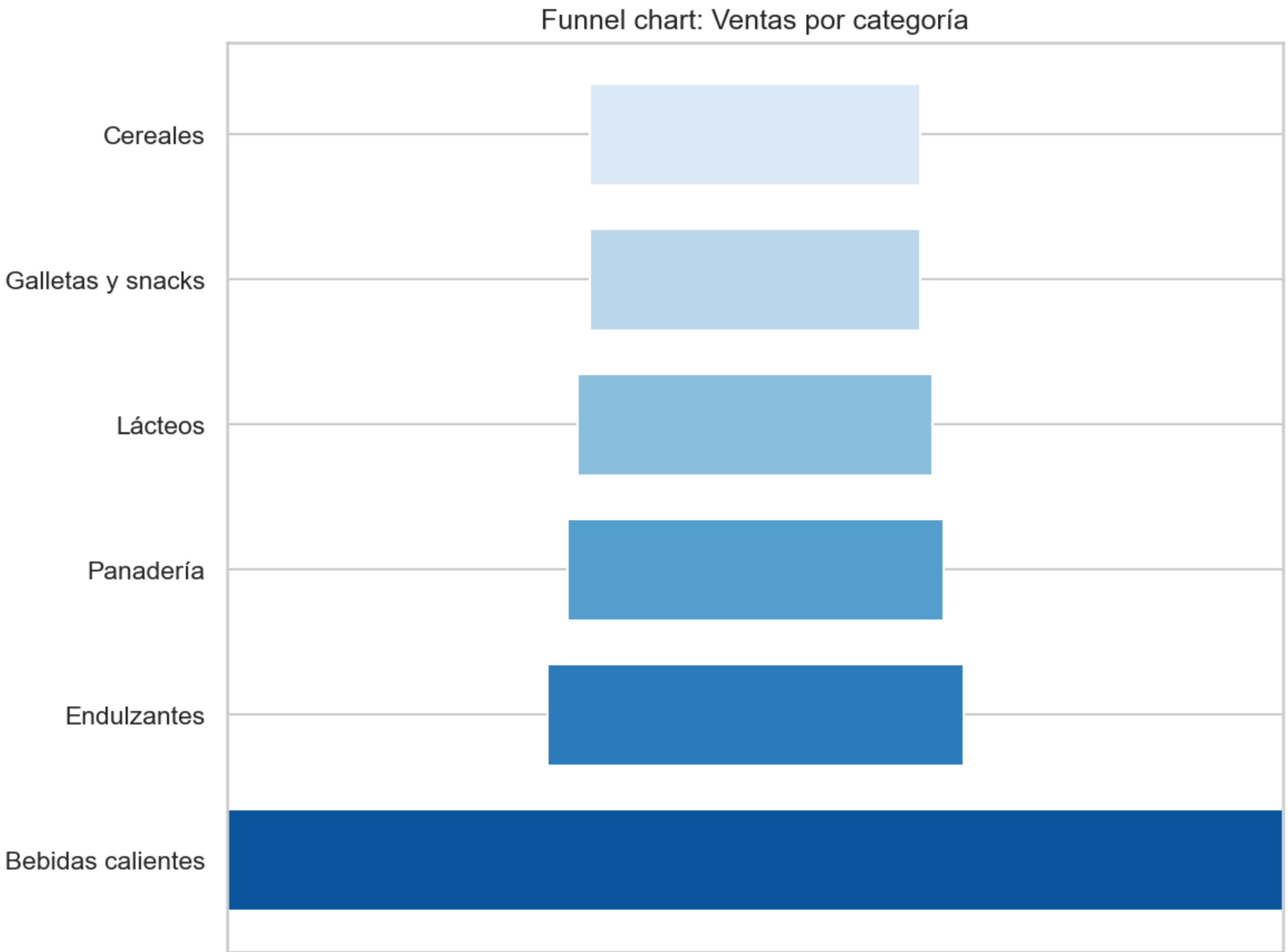

Ventas Totales por Promoción
Promoción
No
Sí
231,483
222,200
0
50000
100000
150000
200000
Ventas Totales

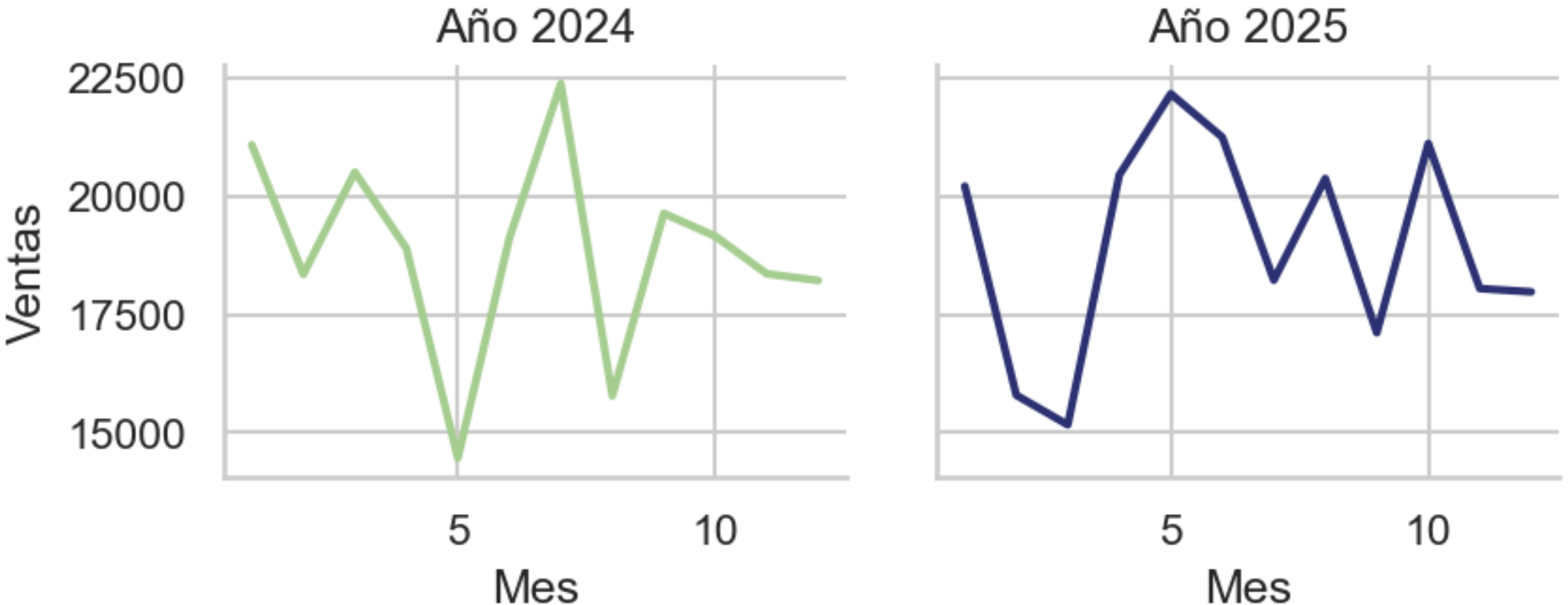

Análisis de Dispersión de Precios Unitarios
Precio Unitario
70
60
50
40
30
20
10
Leche
Galletas
Té
Cereal
Café
Pan
Chocolate
Azúcar
Categoría de Producto

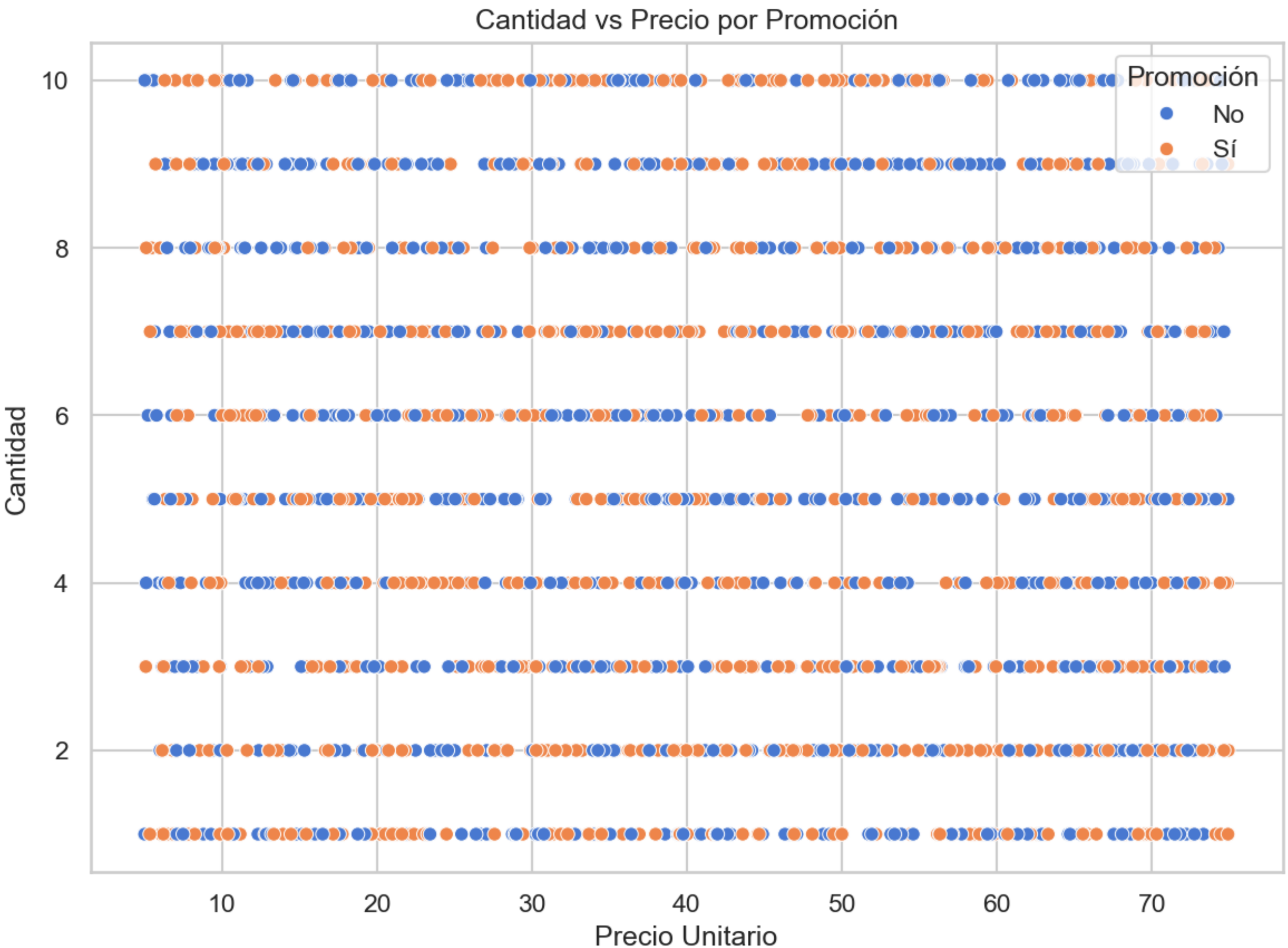

Desempeño por Vendedor y Producto
Vendedor
Ana
Carlos
Jorge
Luis
María
Sofía
Producto
Azúcar
Café
Cereal
Chocolate
Galletas
Leche
Pan
Té
10483.7 9803.1 5387.2 7438.9 10711.5 12065.8 9089.0 9501.4
12339.0 10449.8 12147.6 9487.3 6392.7 9119.3 10700.5 10314.6
9891.0 12567.9 8510.3 6759.0 8575.9 10049.8 10336.5 8625.4
14533.4 8533.4 7034.3 8361.5 9194.9 9867.7 9713.2 6963.6
9372.4 11534.8 9052.5 9039.1 8030.3 6774.1 8537.9 6078.5
9289.8 9376.7 10159.2 11653.1 9434.0 8402.1 11165.8 10833.5
14000
12000
10000
8000
6000

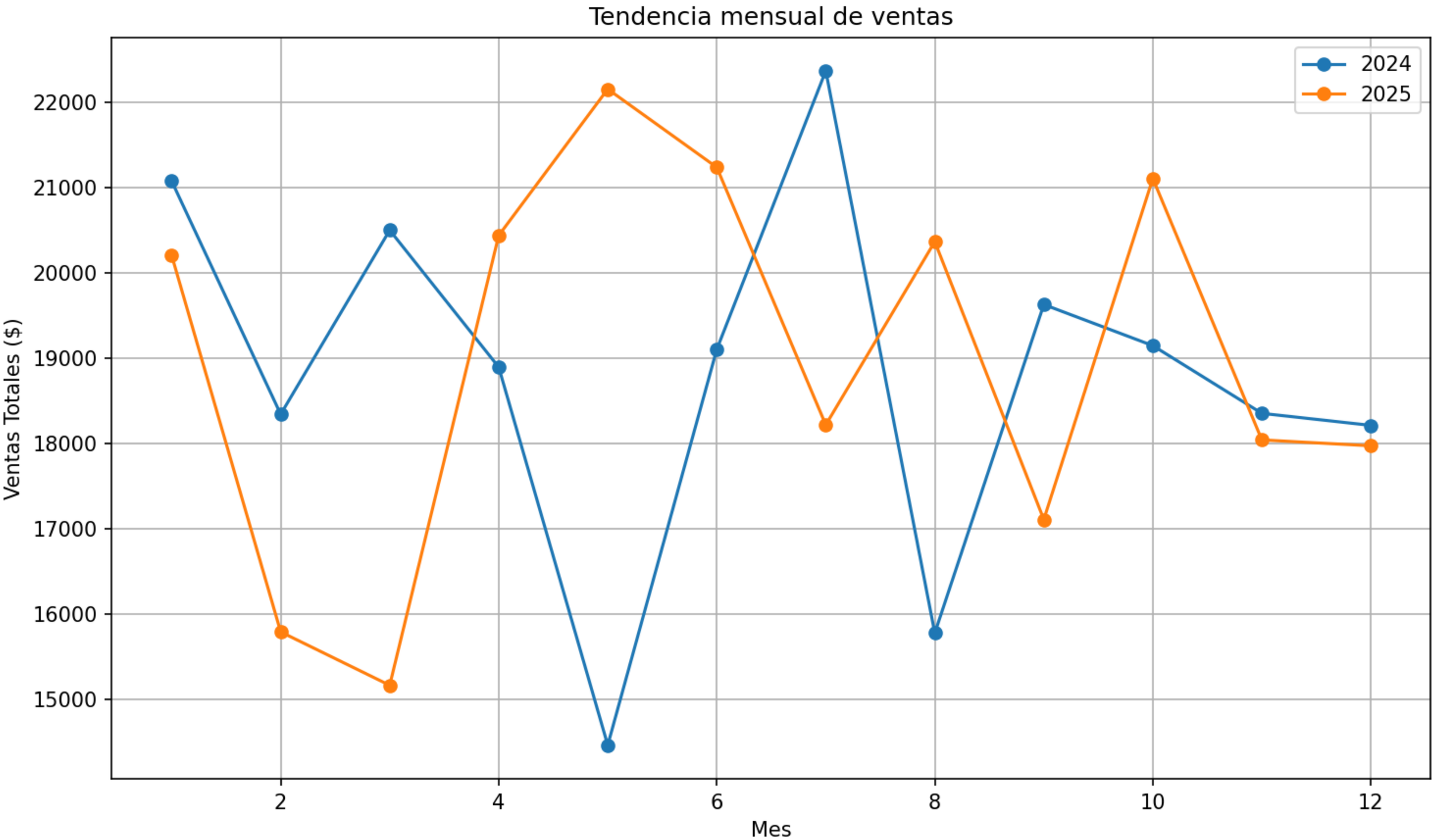

Distribución de Frecuencia de Ventas Totales
Count
0
50
100
150
200
250
300
0
100
200
300
400
500
600
700
Monto de Venta ($)

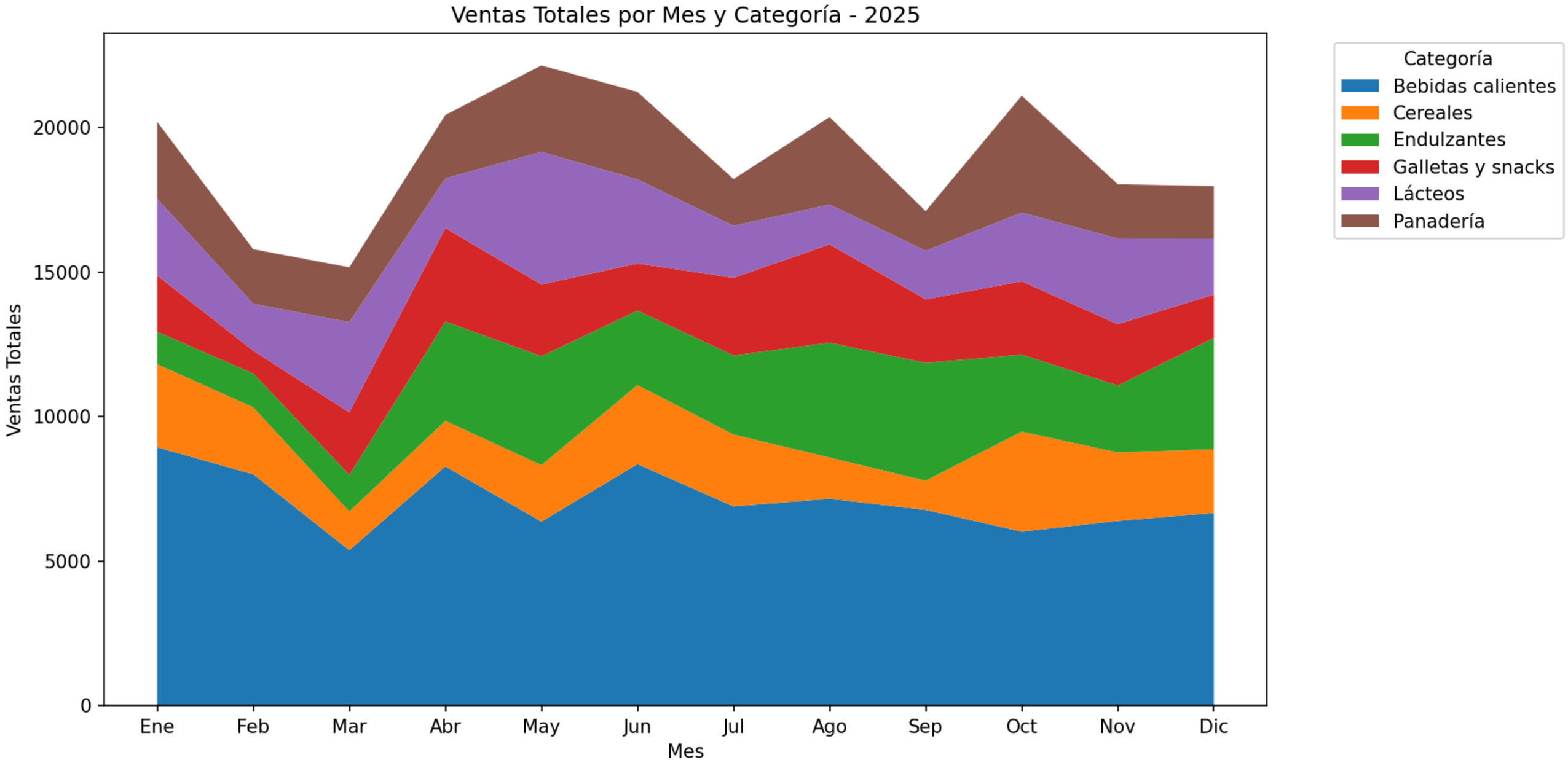

Residuos: Ventas vs Precio Unitario
300
200
100
0
−100
−200
−300
Residuos
10
20
30
40
50
60
70
Precio Unitario

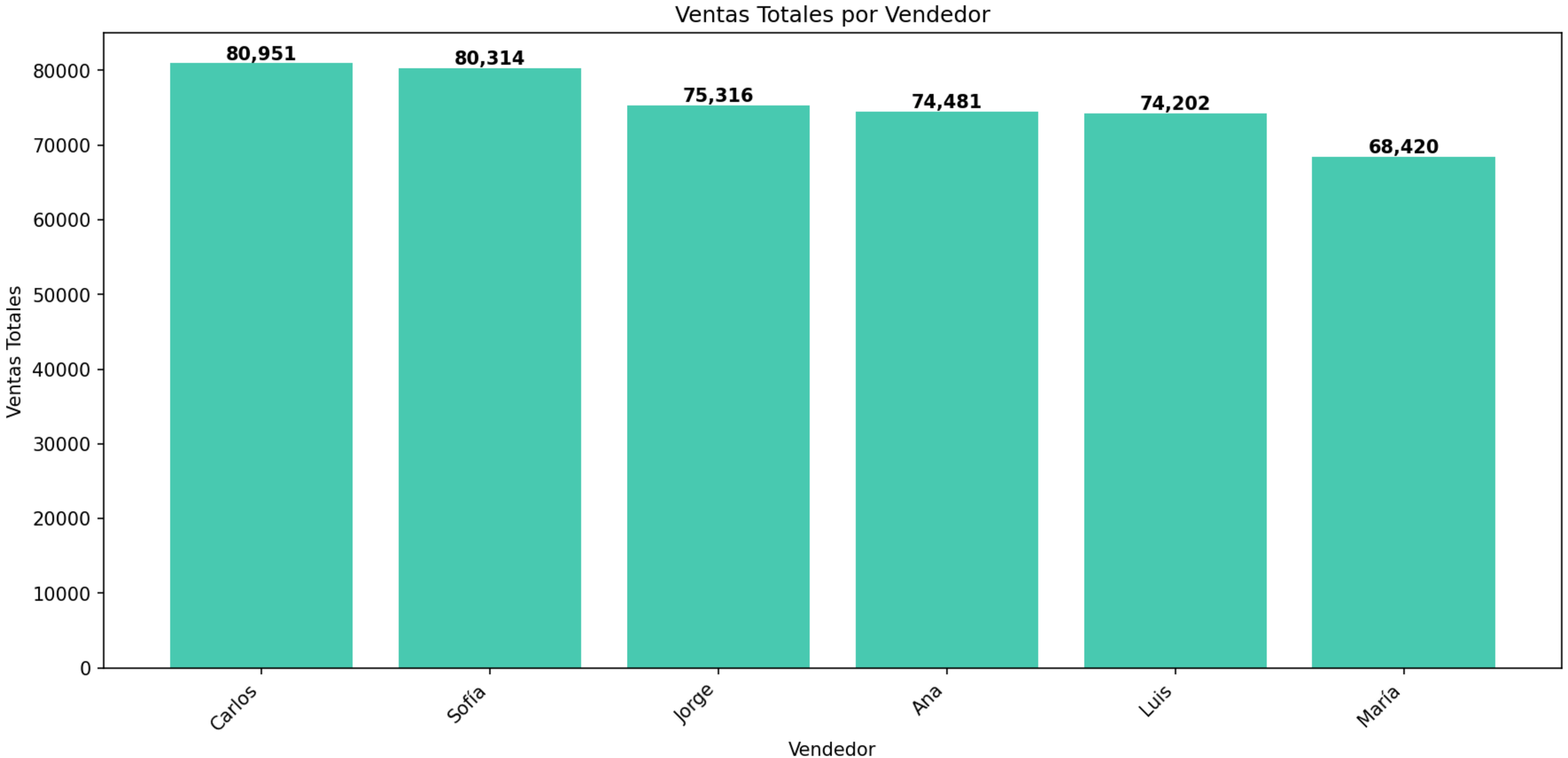

Funnel chart: Ventas por categoría
Cereales
Galletas y snacks
Lácteos
Panadería
Endulzantes
Bebidas calientes

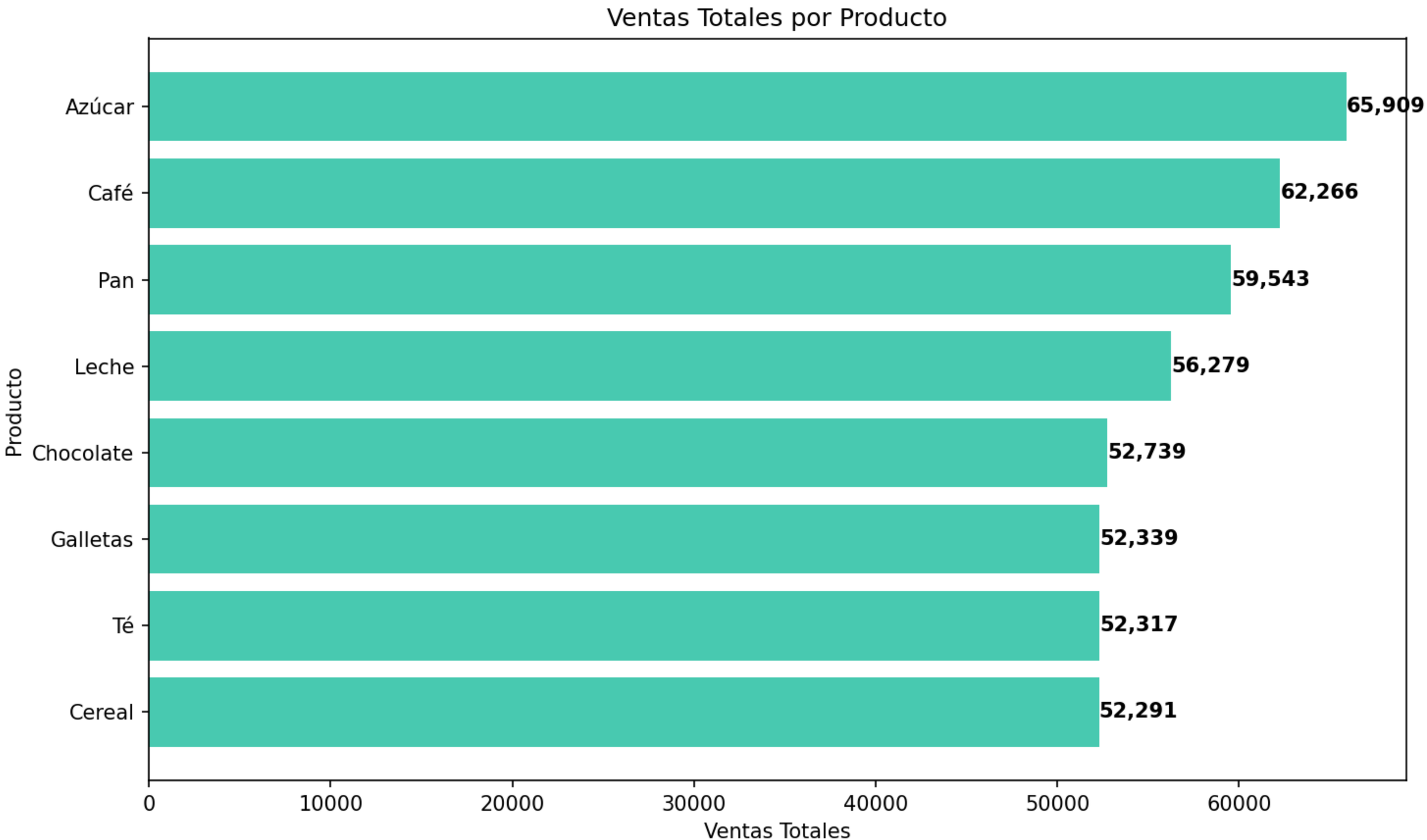

Año 2024
Año 2025
22500
20000
17500
15000
Ventas
5
10
Mes
5
10
Mes

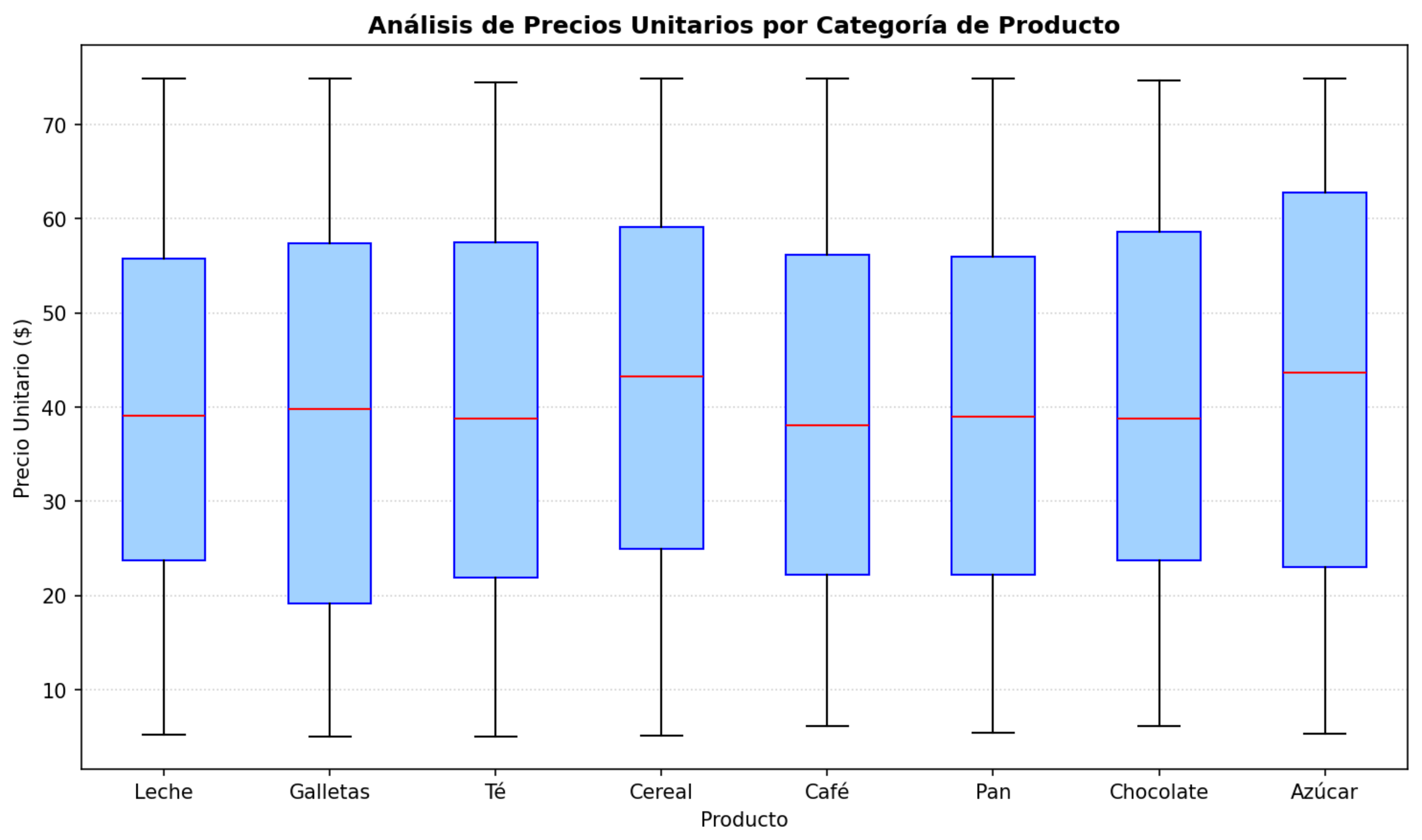

Cantidad vs Precio por Promoción
Promoción
No
Sí
Cantidad
10
8
6
4
2
10
20
30
40
50
60
70
Precio Unitario

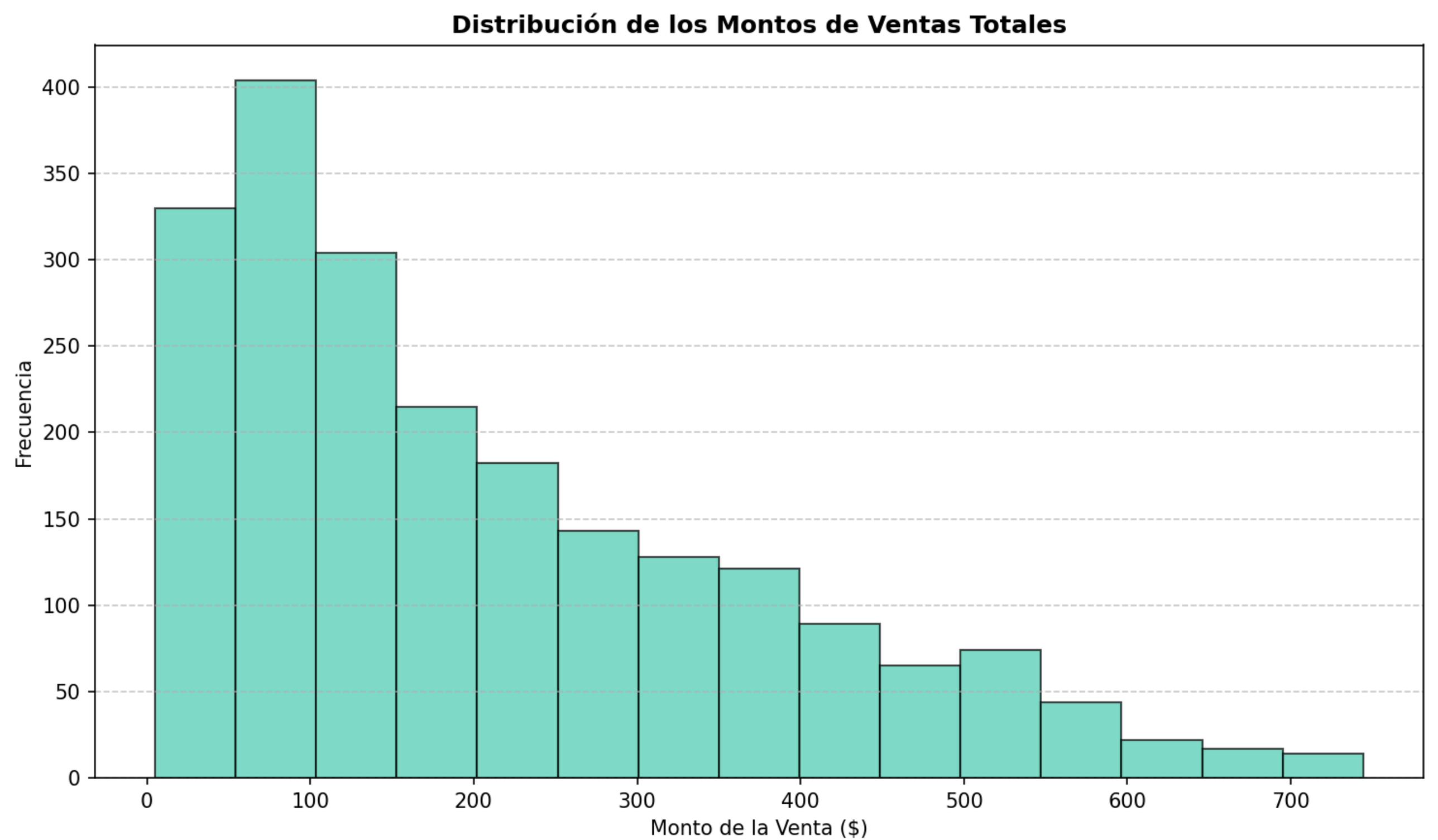

Tendencia mensual de ventas
2024
2025
Ventas Totales ($)
22000
21000
20000
19000
18000
17000
16000
15000
2
4
6
8
10
12
Mes

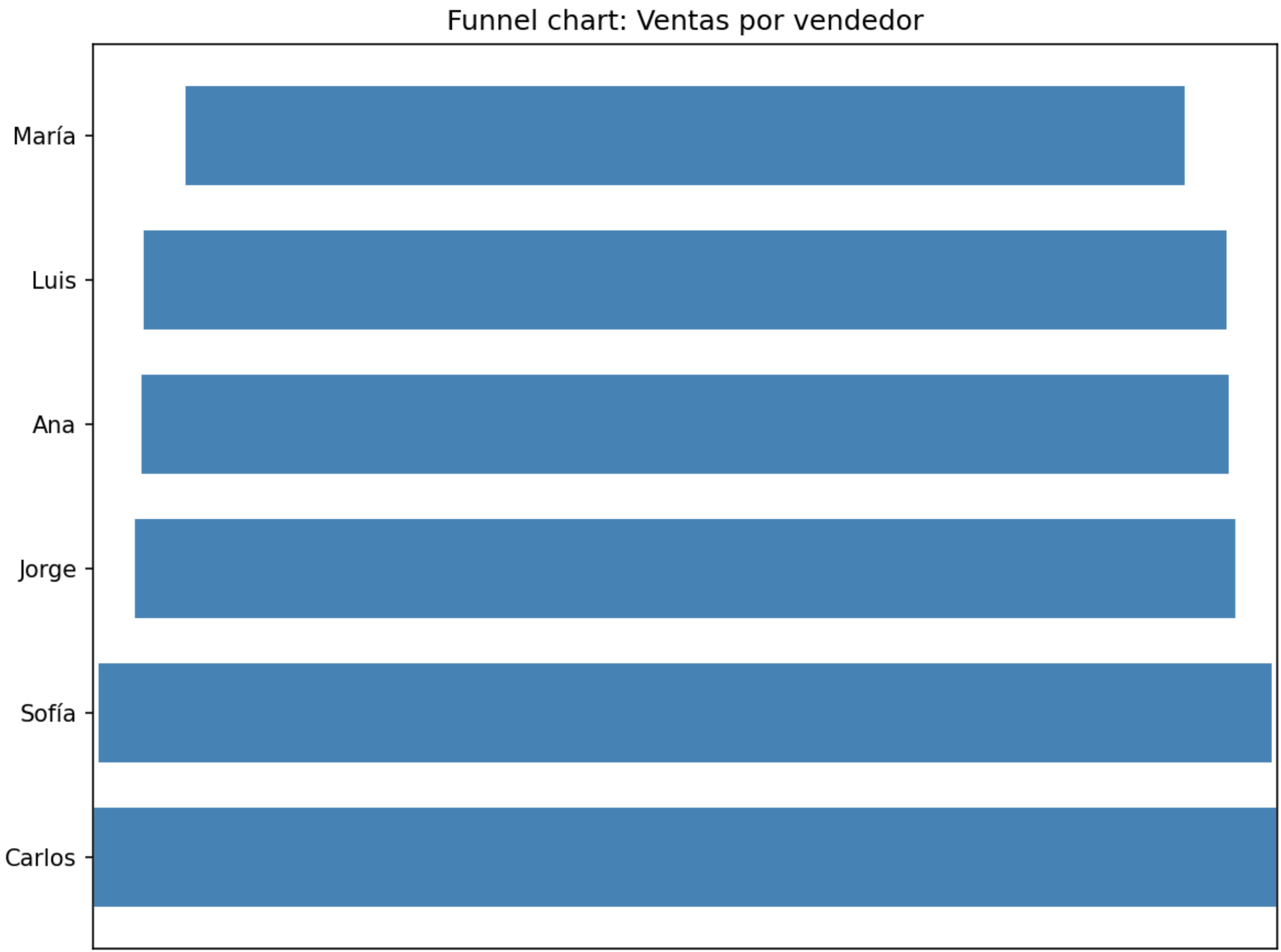

Ventas Totales por Mes y Categoría - 2025
Ventas Totales
0
5000
10000
15000
20000
Ene
Feb
Mar
Abr
May
Jun
Jul
Ago
Sep
Oct
Nov
Dic
Mes
Categoría
Bebidas calientes
Cereales
Endulzantes
Galletas y snacks
Lácteos
Panadería


Ventas Totales por Vendedor
80,951
80,314
75,316
74,481
74,202
68,420
80000
70000
60000
50000
40000
30000
20000
10000
0
Ventas Totales
Carlos
Sofía
Jorge
Ana
Luis
María
Vendedor


Ventas Totales por Producto
Azúcar
Café
Pan
Leche
Chocolate
Galletas
Té
Cereal
65,909
62,266
59,543
56,279
52,739
52,339
52,317
52,291
Producto
0
10000
20000
30000
40000
50000
60000
Ventas Totales


Análisis de Precios Unitarios por Categoría de Producto
Precio Unitario ($)
70
60
50
40
30
20
10
Leche
Galletas
Té
Cereal
Café
Pan
Chocolate
Azúcar
Producto


Distribución de los Montos de Ventas Totales
Frecuencia
0
50
100
150
200
250
300
350
400
0
100
200
300
400
500
600
700
Monto de la Venta ($)


Funnel chart: Ventas por vendedor
María
Luis
Ana
Jorge
Sofía
Carlos